

# Supplementary Material for “Code2SPL: Function-Level Semantic Twins for LLM Program Comprehension and Repair”

ANONYMOUS AUTHOR(S)

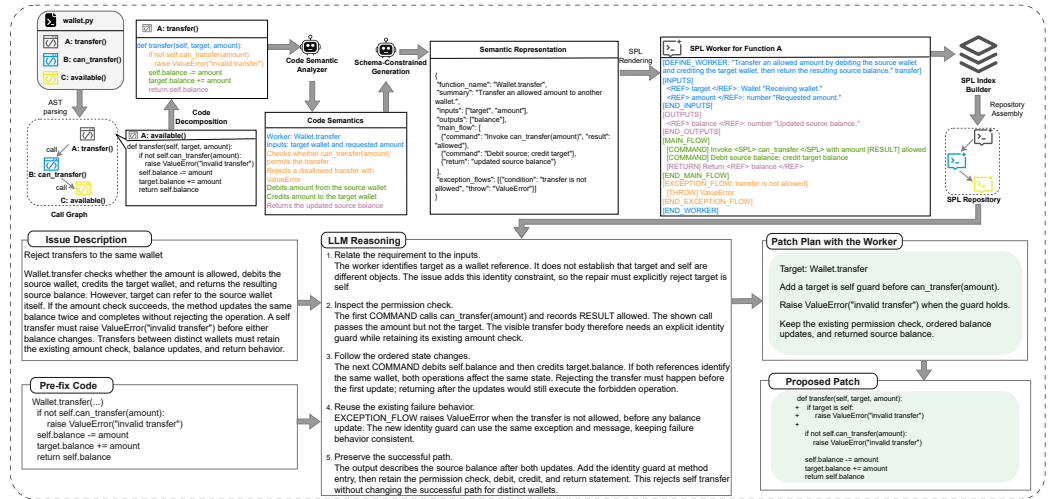
## ACM Reference Format:

Anonymous Author(s). 2027. Supplementary Material for “Code2SPL: Function-Level Semantic Twins for LLM Program Comprehension and Repair”. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (FSE ’27)*. ACM, New York, NY, USA, 22 pages.

This document accompanies the main paper with extended examples, secondary results, protocol details, and resource accounting. References to the main paper retain its section and table numbers.

## A Running Example

Figure A.1 instantiates worker construction for `Wallet.transfer`, then reuses its worker to interpret a new repair requirement.



**Fig. A.1.** Construction and reuse of the SPL worker for `Wallet.transfer`. The upper part instantiates block summarization, schema-constrained generation, template rendering, and indexing. The lower part combines the unchanged worker with a self-transfer requirement, relates its semantic fields to a patch plan, and shows the proposed guard. The reasoning and patch are illustrative, not measured model trajectories.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

FSE '27, Shenzhen, China

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

## A.1 From Source Code to an SPL Worker

The program contains `transfer`, `can_transfer`, and `available`. The call graph places the caller `transfer` above `can_transfer`, which in turn refers to `available`. The bodies of the latter two functions are omitted in the figure, so the example does not assume additional details of their implementations. The visible `transfer` body first queries `can_transfer(amount)`, raises `ValueError` when the result disallows the operation, debits the source balance, credits the target balance, and returns the resulting source balance.

Dividing the function into code blocks exposes the check, branch, balance updates, and return for analysis. Block summarization describes their roles in the block-summary panel. The permission call produces a result used by the branch. The exception path precedes both balance updates. The two updates affect different references, which may or may not refer to different objects. The returned value is the balance after both operations. The worker should preserve these facts without asserting that the target is distinct from the source. Its summary therefore describes the implemented transfer sequence and permission check, rather than treating the desired repair as existing behavior.

Schema-constrained generation organizes these descriptions into the JSON record: inputs and outputs, the permission result, ordered commands, and an exception path. Template rendering converts these fields into the worker labeled SPL for Function A, where Function A is `transfer`. Indexing associates the worker with `Wallet.transfer` and its source location and makes it available in the worker index. The figure abbreviates the intermediate object and worker to show their corresponding semantic fields.

## A.2 From a New Requirement to a Patch

The Issue Description requires a self transfer to follow the existing invalid transfer behavior. The Illustrative Reasoning panel is an authored explanation of how SPL fields inform this edit; the Patch Plan with the Worker summarizes the resulting change. That requirement is supplied after construction. The unchanged worker supports three checks relevant to the requirement. First, the visible permission call receives `amount` but not `target`. Second, the worker records debit and credit operations in sequence, which act on the same object when `target` is `self`. Third, the existing failure path raises `ValueError` before any balance mutation. The patch plan therefore places `target is self` at method entry, before `can_transfer(amount)`, and raises the same exception. The Proposed Patch panel shows this insertion while retaining the amount check, updates, and return. The listing below gives the complete illustrative method.

Listing 1. Illustrative repair preserving the existing successful transfer sequence.

```
def transfer(self, target, amount):
    if target is self:
        raise ValueError("invalid transfer")
    if not self.can_transfer(amount):
        raise ValueError("invalid transfer")
    self.balance -= amount
    target.balance += amount
    return self.balance
```

For ordinary numeric balances, debit and credit on the same object can cancel; that observation does not make self transfer satisfy the new requirement. The rationale for rejecting it is the issue, and the rationale for placing the guard before mutation is the recorded execution order. The illustrated requirement concerns object identity and the placement of the guard in the visible sequential implementation.

## B SPL Access Details

### B.1 Operations in the Evaluated Workflows

The repository stores function identities and source locations together with textual workers and their indexed descriptions. These fields support several distinct operations. Selecting a record chooses the program unit to inspect. Reading interface, command, and flow fields provides the behavioral evidence for that unit. Associating the record with source gives the LLM that uses it a concrete location for checking an expression or making an edit. The final model input contains selected text, but the preceding operations determine which text and program units participate in the task.

In RQ1, class membership fixes the worker set. SPL Only has no original method bodies or official skeleton, so reconstruction tests use of the representation as the behavioral input. The combined condition adds exact interfaces. These experiments use complete task representations rather than a relevance search across an unrestricted repository.

In RQ2, the selected program and reasoning question are fixed. The reasoning LLM uses the corresponding worker with numbered source evidence. References to command results help organize candidate relations, but the evaluator’s definition determines whether a dependency is present. A descriptive mention of a variable is not itself sufficient evidence of a dependency. The exact access rules are documented in Appendix D.

In RQ3, issue text scores records in the prepared index. The selected function views identify candidates and are associated with actual source excerpts. Localization uses them before ordinary repair; Repair starts from a selected function and uses its semantics during editing; Both connects those stages with instructions to avoid redundant investigation. Semantic assets are read only during an attempt. The agent inspects and changes repository code, and the official harness evaluates the resulting patch.

### B.2 Granularity and Reference Use

The representation distinguishes a summary from interfaces and detailed execution paths. Construction already uses summaries and interfaces from available callees rather than copying every complete worker into caller analysis. During use, this distinction provides a basis for inspecting the detail appropriate to a task. A question about a return value may need the commands producing that result and their controlling paths. A repair affecting state may need mutation order and exceptional exits. A complete reconstruction needs broader path coverage.

Validated function references allow an available callee to be identified from a caller’s worker. They provide a route to related semantics without requiring every callee description to be duplicated.

## C Secondary Reconstruction Measures

The method test pass rate measures partial behavioral recovery over the 410 method test units. The CodeBLEU-compatible score is the equally weighted average of token BLEU, weighted BLEU, syntactic AST match, and semantic data-flow match. It provides a structural comparison alongside the primary executable class result. Table ?? reports this score together with executable results; Skeleton + SPL-Prose appears only in the tables of this appendix, and its CodeBLEU is not reported.

These cases also explain why structural resemblance is only a secondary measure. SPL Only slightly exceeds the combined condition on the CodeBLEU-compatible score for several model arms, despite lower executable correctness. Passing tests remains the primary evidence of recovered behavior; structural similarity supplies a complementary diagnostic.

**Table C.1.** Worker representation control on the common 95-task subset. Prose rendering deterministically rewrites the same frozen worker content without rebuilding them. Values are strict correctness percentages;  $p$  is the two-sided exact McNemar value for tagged versus prose within the displayed condition.

| Model         | SPL Only |       |          | Skeleton + SPL |       |       |
|---------------|----------|-------|----------|----------------|-------|-------|
|               | Tagged   | Prose | $p$      | Tagged         | Prose | $p$   |
| DS V4 Flash   | 89.5     | 65.3  | < 0.0001 | 91.6           | 92.6  | 1.000 |
| DS V4 Pro     | 86.3     | 77.9  | 0.0215   | 93.7           | 94.7  | 1.000 |
| GPT-5.4       | 78.9     | 69.5  | 0.064    | 91.6           | 90.5  | 1.000 |
| GPT-5.4 Mini  | 74.7     | 60.0  | 0.0026   | 88.4           | 88.4  | 1.000 |
| Claude Opus 5 | 78.9     | 76.8  | 0.774    | 82.1           | 78.9  | 0.453 |

**Table C.2.** Class-level pass rates (%) over all 100 ClassEval tasks, with 95% Wilson intervals. Paired comparisons are reported separately on the common 95-task subset with evaluable reference implementations.

| Model         | Skeleton        | Free Summary    | SPL-Prose       | SPL Only        | Skeleton + SPL-Prose | Skeleton + SPL  |
|---------------|-----------------|-----------------|-----------------|-----------------|----------------------|-----------------|
| DS V4 Flash   | 46 [36.6, 55.7] | 25 [17.5, 34.3] | 62 [52.2, 70.9] | 85 [76.7, 90.7] | 88 [80.2, 93.0]      | 87 [79.0, 92.2] |
| DS V4 Pro     | 50 [40.4, 59.6] | 33 [24.6, 42.7] | 75 [65.7, 82.5] | 83 [74.5, 89.1] | 91 [83.8, 95.2]      | 89 [81.4, 93.7] |
| GPT-5.4       | 43 [33.7, 52.8] | 53 [43.3, 62.5] | 66 [56.3, 74.5] | 75 [65.7, 82.5] | 87 [79.0, 92.2]      | 87 [79.0, 92.2] |
| GPT-5.4 Mini  | 43 [33.7, 52.8] | 50 [40.4, 59.6] | 58 [48.2, 67.2] | 71 [61.5, 79.0] | 85 [76.7, 90.7]      | 85 [76.7, 90.7] |
| Claude Opus 5 | 58 [48.2, 67.2] | 47 [37.5, 56.7] | 74 [64.6, 81.6] | 76 [66.8, 83.3] | 76 [66.8, 83.3]      | 79 [70.0, 85.8] |

**Table C.3.** Supplementary method-level pass rates (%) over 410 test units. These quantify partial reconstruction; PASS in the main paper requires all tests for a class to pass.

| Model         | Skeleton | Free Summary | SPL Only | Skeleton + SPL | SPL-Prose | Skeleton + SPL-Prose |
|---------------|----------|--------------|----------|----------------|-----------|----------------------|
| DS V4 Flash   | 71.5     | 51.7         | 92.9     | <b>95.4</b>    | 76.1      | <b>95.4</b>          |
| DS V4 Pro     | 79.5     | 59.0         | 87.8     | 95.6           | 83.2      | <b>96.3</b>          |
| GPT-5.4       | 76.1     | 73.2         | 82.7     | <b>94.9</b>    | 77.8      | 92.9                 |
| GPT-5.4 Mini  | 75.4     | 71.7         | 82.2     | 92.4           | 73.2      | <b>92.9</b>          |
| Claude Opus 5 | 79.8     | 73.2         | 79.3     | <b>82.9</b>    | 78.8      | 79.8                 |

*Illustrative semantic requirements.* BookManagementDB requires exact method identities, SQL effects, cursor state, and tuple-valued query results. A high-level description of database management does not by itself specify that contract. ArrangementCalculator illustrates why static-method declarations matter to an executable interface. AccessGatewayFilter illustrates the need to preserve an implicit None return rather than replacing it with a Boolean interpretation.

*Interfaces and dependency compatibility.* The official ClassEval skeleton supplies concrete import and API cues, reducing the need to infer how an implementation is exposed to its environment. This complements the behavior supplied by SPL. Environment-sensitive APIs also affect reconstruction, including removal of np.mat in NumPy 2.0. Such compatibility differences help explain why reference behavior and executable outcomes require separate checks; they do not by themselves identify the cause of the Skeleton + SPL improvement. The API audit below directly measures the interface-preservation interpretation.

## C.1 Five-model API fidelity audit

Table C.4 compares generated APIs with the reference implementation using AST-based checks. The columns successively require the class name, method set, parameter and static-method shape, and decorators to match. The audit complements execution because a class can describe the right functionality while changing the interface through which tests and downstream code reach that behavior.

**Table C.4.** API fidelity over 100 ClassEval tasks. “Shape” includes parameter structure and static/class method kind; “+ decorators” additionally requires decorator agreement.

| Model        | Condition      | Class | Methods | Shape | + decorators |
|--------------|----------------|-------|---------|-------|--------------|
| Flash        | Skeleton       | 90    | 87      | 85    | 85           |
|              | Free Summary   | 97    | 52      | 19    | 19           |
|              | SPL Only       | 100   | 98      | 89    | 89           |
|              | Skeleton + SPL | 99    | 97      | 95    | 95           |
| Pro          | Skeleton       | 100   | 96      | 94    | 94           |
|              | Free Summary   | 93    | 67      | 35    | 35           |
|              | SPL Only       | 100   | 99      | 82    | 82           |
|              | Skeleton + SPL | 99    | 98      | 96    | 96           |
| GPT-5.4      | Skeleton       | 100   | 95      | 92    | 92           |
|              | Free Summary   | 97    | 89      | 76    | 76           |
|              | SPL Only       | 100   | 99      | 83    | 83           |
|              | Skeleton + SPL | 100   | 99      | 97    | 97           |
| GPT-5.4 Mini | Skeleton       | 100   | 95      | 92    | 92           |
|              | Free Summary   | 99    | 91      | 73    | 73           |
|              | SPL Only       | 100   | 99      | 79    | 79           |
|              | Skeleton + SPL | 99    | 98      | 96    | 96           |
| Opus 5       | Skeleton       | 94    | 92      | 90    | 90           |
|              | Free Summary   | 98    | 78      | 41    | 41           |
|              | SPL Only       | 89    | 85      | 78    | 78           |
|              | Skeleton + SPL | 86    | 85      | 84    | 84           |

## D Evaluation Protocol and Statistical Details

### D.1 Samples, Settings, and Denominators

For the two sampled experiments, we determine the target sample size using the finite population calculation

$$n = \frac{Nz^2p(1-p)}{\epsilon^2(N-1) + z^2p(1-p)}, \quad (1)$$

where  $N$  is the evaluation population,  $z = 1.96$  corresponds to a 95% confidence level,  $p = 0.5$  gives the conservative maximum variance, and  $\epsilon = 0.05$  specifies a 5% margin of error. The samples contain 341 CoRe instances and 218 SWE-bench Verified tasks. The calculation provides a planning approximation; the frozen identifiers define the actual evaluation samples. Sampling is performed without replacement, with CoRe stratified by task category and mode; the selected identifiers are

**Table D.5.** Execution settings. A model response limit is distinct from total usage over an agent trajectory.

| Setting                   | RQ1                                                     | RQ2                         | RQ3                                 |
|---------------------------|---------------------------------------------------------|-----------------------------|-------------------------------------|
| Worker construction model | Same model as reconstruction                            | DeepSeek V4 Flash           | DeepSeek V4 Flash                   |
| Consumer                  | Five configurations                                     | DeepSeek V4 Pro             | DeepSeek V4 Pro                     |
| Temperature               | 0                                                       | 0                           | 0                                   |
| Response limit            | 4,096 tokens                                            | 16,384 tokens               | 8,192 tokens                        |
| Task interaction limit    | One reconstruction                                      | Recorded reasoning protocol | 36 steps                            |
| Representation reuse      | Shared by tagged and prose conditions within each model | Frozen snapshot             | Frozen assets across SPL conditions |

frozen before model execution. Every condition within the same experiment is evaluated on exactly the same task identifiers.

ClassEval uses all 100 class tasks and 410 method test units. The primary strict results retain all 100 tasks. Paired analyses use the common 95-task subset for which the reference execution is stable under the unified environment; the same exclusion set is applied to all five model arms. This keeps the paired comparisons model independent while preserving a conservative all-task primary denominator.

CoRe samples 341 task and mode instances from the 3,013 instances in the evaluated lightweight configuration. SWE-bench Verified samples 218 tasks from 500. Both samples use seed 20260531 and sampling without replacement. The finite population calculation determines a sample size for estimating a proportion; it is not a power calculation for paired treatment comparisons and does not remove uncertainty caused by model execution. All conditions within an experiment use the same selected identifiers.

The RQ2 configuration enables model reasoning; model usage includes all calls made under the final protocol. Model names identify the requested configurations; gateway deployment details are not independently verified. Flash uses a different frozen summary prompt from the other reconstruction arms, so cross-model summary rankings are descriptive. Primary inferences use within-model paired comparisons. Each task has one final outcome per condition; the paired tests quantify task-level differences and do not estimate repeated-decoding variance.

## D.2 Paired Comparisons

**Table D.6.** Paired repair comparisons against Original. Exact probabilities are two-sided exact McNemar values from the discordant counts; Holm values adjust within the four comparisons against Original.

| Condition        | Gain | Loss | Net | Exact $p$ | Holm $p$ |
|------------------|------|------|-----|-----------|----------|
| Free Summary     | 17   | 13   | +4  | 0.585     | 0.585    |
| SPL Localization | 35   | 8    | +27 | 0.000042  | 0.000126 |
| SPL Repair       | 36   | 7    | +29 | 0.000009  | 0.000036 |
| SPL Both         | 35   | 8    | +27 | 0.000042  | 0.000126 |

For each comparison, a gain is an instance answered correctly or resolved only by the first condition, and a loss is an instance answered correctly or resolved only by the comparator. Two-sided exact McNemar tests use these discordant pairs. RQ1 applies Holm correction within each

**Table D.7.** Overall CoRe paired outcomes. The first three rows form one Holm family. The final three compare SPL-Prose with each existing condition and form a second family.

| Comparison                         | Gain | Loss | Net | Exact $p$ | Holm $p$      |
|------------------------------------|------|------|-----|-----------|---------------|
| Structured SPL versus Raw          | 30   | 14   | +16 | 0.0226    | 0.0679        |
| Structured SPL versus Free Summary | 30   | 16   | +14 | 0.0541    | 0.1082        |
| Free Summary versus Raw            | 14   | 12   | +2  | 0.8450    | 0.8450        |
| SPL-Prose versus Raw               | 13   | 16   | -3  | 0.7111    | 0.8097        |
| SPL-Prose versus Free Summary      | 9    | 14   | -5  | 0.4049    | 0.8097        |
| SPL-Prose versus Structured SPL    | 13   | 32   | -19 | 0.0066    | <b>0.0198</b> |

model to the four comparisons of the two SPL conditions with Skeleton and Free Summary. On the common 95-task subset, all 20 planned within-model comparisons remain significant after the model-specific Holm correction. For Claude Opus 5, Skeleton + SPL has 25 gains and 5 losses against Skeleton on the common 95-task subset, yielding 20 additional passing classes and a 21.1 percentage-point improvement. This paired comparison uses the 95-task subset, whereas Table C.2 reports results over all 100 tasks.

RQ2 reports all three overall pairwise contrasts. Table D.7 reports these contrasts with Holm adjustment over the three of them. Structured SPL versus Raw has exact  $p = 0.0226$  and Holm-adjusted  $p = 0.068$ , so the overall difference is not significant after correction. The three category comparisons with Raw form a separate Holm family. Their gain and loss counts are 22 and 8 for information flow, 5 and 2 for data dependency, and 3 and 4 for control dependency, with Holm-adjusted  $p$  values of 0.048, 0.906, and 1.000. Only information flow remains significant in that category family.

For RQ3, the four comparisons of Free Summary, SPL Localization, SPL Repair, and SPL Both against Original form one Holm family (Table D.6). All adjusted values are computed from the discordant counts shown in the table. SPL-Prose is not part of the paired analysis.

Both versus Free Summary has 29 gains and 6 losses (exact  $p = 0.000117$ ); this comparison is outside the Holm family. Among SPL variants, Repair versus Localization has 9 gains and 7 losses; Both versus Localization has 9 gains and 9 losses; Both versus Repair has 8 gains and 10 losses. Their exact probabilities range from 0.8036 to 1.0000. These comparisons do not identify a significantly better SPL placement, and failure to reject their differences does not establish equivalence.

**D.3 CoRe Category Results and Consumption Rules**

The final access protocol retains numbered source and checks candidate dependencies against the official task definitions. In counted loops, control predicates are distinguished from data-producing expressions; in Python iteration, the iterable expression and explicit read/write operations are distinguished from a new variable binding. Mention in a semantic description is not itself evidence of a dependency.

The evaluation is protocol-dependent and does not constitute an independent held-out validation of the access rules. This limits causal and out-of-sample interpretation without changing the reported final outcomes. Parsing succeeds for 338 Raw, 340 Free Summary, 341 SPL-Prose, and 340 Structured SPL responses out of 341; parseability is not correctness.

The prose renderer converts the supplied SPL view content into ordinary sentences, preserving the audited semantic items while omitting SPL field markers and using the Free Summary prompt framework. Across the 303 paired instances with identical view material, Raw and Free Summary

**Table D.8.** CoRe accuracy by task category and mode (%). Category sizes are 134 for data dependency, 93 for control dependency, and 114 for information flow. Overall correct sets and category totals are reported in the main paper; resource accounting appears below.

**Accuracy by task category and mode (%)**

| Category           | Mode   | Raw  | Free Summary | SPL-Prose | Structured SPL |
|--------------------|--------|------|--------------|-----------|----------------|
| Data dependency    | Source | 74.6 | 68.7         | 65.7      | 77.6           |
|                    | Trace  | 95.5 | 95.5         | 94.0      | 97.0           |
| Control dependency | Source | 86.8 | 94.7         | 92.1      | 89.5           |
|                    | Trace  | 94.5 | 92.7         | 94.5      | 90.9           |
| Information flow   | Source | 35.1 | 38.6         | 35.1      | 50.9           |
|                    | Trace  | 87.7 | 91.2         | 91.2      | 96.5           |

each solve 242, Structured SPL solves 252, and SPL-Prose solves 235. The structured-versus-prose comparison has exact  $p = 0.0076$  and Holm-adjusted  $p = 0.0229$ . The remaining 38 instances have non-identical semantic inputs across the two conditions, so the full-sample contrast is not treated as a strict single-variable ablation.

#### D.4 Repair Context and Supplementary Outcomes

Workers describe the pre-fix revision. The repair protocol explicitly warns that a recorded condition or state update can be the behavior that needs correction. It selects views by issue relevance and associates them with source excerpts. The files available for construction are limited using reference patch file paths; reference edit contents are withheld. Thus candidate selection operates within a scope that already benefits from oracle information.

All three structured SPL variants select up to four views, inline two, limit each view to 750 characters, and limit the source excerpt to 1,400 characters. Their workflow instructions differ by the stage at which workers are used.

The SPL-Prose control uses the same frozen 218-task list and does not rebuild SPL. For each task, every function entry in `spl_index.json` is converted into ordinary sentences in its existing order and appended once to the Original agent template. The appended text has a median size of 10,290 tokens, a mean of 17,576, and a maximum of 104,889. This is an ordinary-summary control: the semantic facts come from the prepared SPL analysis, while the task receives them as one prose block rather than as selectively accessed workers. The condition resolves 100 tasks and produces 120 nonempty patches; all submitted patches are evaluated with the official harness.

**Table D.9.** Retained supplementary repair conditions on 218 tasks. Lifecycle counts are computed from the resolved and generated-patch totals. Inference tokens exclude construction.

| Condition | Resolved (%) | Pass | Fail | No Patch | Tokens (M) |
|-----------|--------------|------|------|----------|------------|
| SPL Both  | 64.7         | 141  | 50   | 27       | 41.31      |
| SPL-Prose | 45.9         | 100  | 20   | 98       | 132.13     |

## E Repair under an 80-Step Interaction Budget

### E.1 Setup and Sample Composition

This supplementary study asks whether the repair advantage persists when the interaction limit increases from 36 to 80 steps. The repair model remains DeepSeek V4 Pro with temperature 0 and

an 8,192-token output limit; all other repair settings and prompt strategies are unchanged. All six conditions use the same task revisions as in RQ3; conditions requiring SPL views or summaries reuse the corresponding frozen assets. A step denotes one model call. Outcomes use the same official repair evaluator and the same three mutually exclusive patch states.

The study uses 30 tasks from the primary 218-task sample, stratified by Original and SPL Repair outcomes at 36 steps (Table E.10). It includes all seven Original-only successes, 12 of the 36 SPL-only successes, and 11 of the 68 joint failures. The latter two strata use fixed seed 12680. The 107 joint successes are omitted to focus on disagreements and unresolved tasks. All six methods use the same selected tasks. Accordingly, the comparisons below describe budget sensitivity on this deliberately selected subset; their rates have a different denominator and sampling composition from the primary 218-task results.

**Table E.10.** Sample composition and 80-step outcomes by original 36-step stratum. Success columns count resolved tasks within the selected stratum.

| 36-step stratum | Available | Selected | Original (80) | SPL Repair (80) |
|-----------------|-----------|----------|---------------|-----------------|
| Original only   | 7         | 7        | 4             | 4               |
| SPL Repair only | 36        | 12       | 6             | 11              |
| Neither         | 68        | 11       | 3             | 2               |
| Both            | 107       | 0        | –             | –               |
| Total           | 218       | 30       | 13            | 17              |

## E.2 Outcomes and Changes from 36 Steps

Table E.11 reports the 80-step lifecycle distribution alongside the 36-step resolved counts on the *same* subset. Condition names follow the main paper; SPL-Prose supplies the complete prose rendering of the frozen SPL index. SPL Both uses the primary configuration.

**Table E.11.** Repair outcomes on the same 30 tasks under 36- and 80-step interaction limits. Patch Pass equals the resolved count; Patch Fail denotes an evaluated patch that does not resolve the task; No Patch denotes no valid patch generated.

| Condition        | 36 steps | 80 steps   |            |          | Resolved (%) |
|------------------|----------|------------|------------|----------|--------------|
|                  | Resolved | Patch Pass | Patch Fail | No Patch |              |
| Original         | 7        | 13         | 10         | 7        | 43.3         |
| Free Summary     | 9        | 17         | 7          | 6        | 56.7         |
| SPL Localization | 15       | <b>19</b>  | 10         | 1        | <b>63.3</b>  |
| SPL Repair       | 12       | 17         | 9          | 4        | 56.7         |
| SPL Both         | 14       | 18         | 9          | 3        | 60.0         |
| SPL-Prose        | 12       | 12         | 9          | 9        | 40.0         |

At 80 steps, SPL Localization resolves 19 tasks, followed by SPL Both with 18 and SPL Repair and Free Summary with 17 each. Relative to Original’s 13 successes, Localization adds six (20.0 percentage points) and Repair adds four (13.3 points). Localization produces patches for 29/30 tasks, compared with 23/30 for Original; Repair produces patches for 26/30. Thus the broader patch production seen in the primary experiment also appears under the relaxed budget.

**Table E.12.** Task-level transitions from 36 to 80 steps. Gains are unresolved-to-resolved transitions; losses are resolved-to-unresolved transitions. Retained successes are correct at both budgets.

| Condition        | Retained | Gains | Losses | Net change |
|------------------|----------|-------|--------|------------|
| Original         | 4        | 9     | 3      | +6         |
| Free Summary     | 7        | 10    | 2      | +8         |
| SPL Localization | 15       | 4     | 0      | +4         |
| SPL Repair       | 11       | 6     | 1      | +5         |
| SPL Both         | 14       | 4     | 0      | +4         |
| SPL-Prose        | 8        | 4     | 4      | 0          |

The transitions in Table E.12 reveal different responses to the additional budget. On the same 30 tasks, Original and Free Summary resolve six and eight more tasks, respectively, under the 80-step limit. SPL Repair improves by five and still exceeds Original, while Free Summary matches Repair on this subset. Localization and Both retain every 36-step success and add four each. SPL-Prose remains at 12 successes, but this unchanged total conceals four gains and four losses. Budget expansion therefore changes which tasks succeed, as well as the aggregate count; the 80-step runs are independent executions, not continuations of the 36-step trajectories.

The stratified view further explains the residual difference. Original resolves six of the 12 previously SPL-only tasks at 80 steps, whereas Repair retains 11. Repair also resolves four of the seven previously Original-only tasks. In the joint-failure stratum, Original resolves three and Repair two. The observed advantage consequently comes from different coverage across strata, rather than a uniform benefit on every task group.

### E.3 Resource Use and Interpretation

Table E.13 reports final inference usage; construction is not repeated. SPL Localization uses 6.71 million tokens versus 24.96 million for Original, a 73.1% reduction. SPL Repair and Both use 7.22 and 7.74 million, reductions of 71.1% and 69.0%. Their mean calls per task are 13.7, 15.5, and 16.4, compared with 36.7 for Original and 35.8 for Free Summary. These values show that the SPL conditions use substantially less of the available interaction budget while resolving more tasks than Original.

**Table E.13.** 80-step inference resources across 30 tasks per condition. Totals are input plus output tokens; tokens per resolved task divide total usage, including unsuccessful tasks, by the resolved count.

| Condition        | Calls | Input tokens | Output tokens | Total tokens | Tokens/resolved |
|------------------|-------|--------------|---------------|--------------|-----------------|
| Original         | 1,102 | 24,079,009   | 876,729       | 24,955,738   | 1,919,672       |
| Free Summary     | 1,074 | 20,690,502   | 715,890       | 21,406,392   | 1,259,200       |
| SPL Localization | 412   | 6,362,546    | 345,729       | 6,708,275    | 353,067         |
| SPL Repair       | 466   | 6,832,632    | 388,086       | 7,220,718    | 424,748         |
| SPL Both         | 491   | 7,285,158    | 455,839       | 7,740,997    | 430,055         |
| SPL-Prose        | 728   | 26,721,831   | 618,627       | 27,340,458   | 2,278,372       |

SPL-Prose uses 27.34 million tokens and resolves 12 tasks. Its larger semantic context does not reproduce the outcomes or inference efficiency of the structured conditions. Together, the results support using recovered semantics through source-linked, selectively accessed units. Increasing the interaction budget improves the baselines but does not remove SPL’s advantage over Original on the selected tasks. On this same subset, Localization has the highest resolved count at both budgets. Its margin over Original narrows from eight to six tasks, while Repair’s margin narrows from

five to four. Thus the extra budget closes part of the baseline gap, while source-linked semantic guidance remains useful at both evaluated limits.

#### E.4 Actual Interaction Steps and Token Demand

An interaction limit is an upper bound, not a prescribed trajectory length. We measure mean actual steps as  $\bar{s} = \sum_i c_i / N$ , where  $c_i$  is the number of model calls for task  $i$ . Mean inference tokens are  $\bar{t} = \sum_i (t_i^{\text{in}} + t_i^{\text{out}}) / N$ . Both averages include unsuccessful tasks and exclude representation construction. A tool invocation is not counted as an additional model call.



**Fig. E.2.** Mean actual interaction steps and inference tokens per task under the two interaction limits. The 36-step series covers the 218-task main cohort; the 80-step series covers the selected 30-task cohort. Lines aid comparison across methods, not interpolation. The different cohorts preclude interpreting the vertical gap as a paired effect of increasing the limit.

Figure E.2 separates interaction frequency from token demand. At the 36-step limit, Original and Free Summary average 24.05 and 24.27 steps, whereas SPL Localization, SPL Repair, and SPL Both average 13.89, 14.81, and 13.83. Their mean token usage is 196.57k, 211.25k, and 189.50k, versus 350.42k for Original. SPL Both therefore uses 42.5% fewer model calls and 45.9% fewer inference tokens than Original on the same 218 tasks, while resolving 141 rather than 114.

At the 80-step limit, the three structured conditions average 13.73–16.37 steps, while Original and Free Summary average 36.73 and 35.80. Thus the larger allowance does not force longer trajectories: actual interaction demand remains method-dependent. SPL-Prose makes fewer calls than Original in each cohort (18.98 versus 24.05; 24.27 versus 36.73), yet uses more tokens per task (606.12k versus 350.42k; 911.35k versus 831.86k). Fewer calls alone therefore do not establish lower inference cost; the amount of context processed per call also matters.

The within-cohort comparisons support lower interaction and token demand for structured access. The cross-cohort lines are descriptive: paired resource changes require the 36-step usage records for the same 30 selected tasks. Paired outcome changes on those tasks are reported in Table E.12.

F Resource Consumption

F.1 Accounting Boundary

Construction produces reusable representations; inference uses those representations for a task. The following tables keep these stages separate. SPL Only, Skeleton + SPL, and the two prose-rendering controls share one construction cost within each reconstruction model. The two SPL-based reasoning conditions share one SPL asset; deterministic prose rendering adds no model call. Shared construction must not be counted twice when aggregating conditions.

Token usage records accumulated model input and output across calls. Reasoning tokens already included in output usage are not added again. Repeated input in an interactive trajectory contributes repeatedly to the total; the total is not the size of the unique source or semantic repository. Token reductions are not direct estimates of monetary cost, computation, energy, or latency. Cross-provider token totals are descriptive because tokenizers and accounting interfaces can differ.

F.2 Experiment 1: Construction and Reconstruction

Table F.14. ClassEval construction over 100 tasks per model. Each SPL total is counted once and shared by the tagged and prose SPL conditions. Model names follow the current five-configuration evaluation.

| Model         | Worker construction |           | Summary construction |         |
|---------------|---------------------|-----------|----------------------|---------|
|               | Calls               | Tokens    | Calls                | Tokens  |
| DS V4 Flash   | 1,124               | 4,416,380 | 109                  | 105,800 |
| DS V4 Pro     | 1,124               | 4,365,026 | 100                  | 63,718  |
| GPT-5.4       | 1,124               | 1,576,145 | 100                  | 70,342  |
| GPT-5.4 Mini  | 1,124               | 1,454,868 | 100                  | 69,269  |
| Claude Opus 5 | 1,124               | 4,306,271 | 100                  | 148,987 |

Table F.15. ClassEval inference tokens by condition. Totals exclude construction.

| Model         | Skeleton  | Free Summary | SPL Only | Skeleton + SPL | Total     |
|---------------|-----------|--------------|----------|----------------|-----------|
| DS V4 Flash   | 1,200,063 | 862,527      | 872,987  | 1,030,819      | 3,966,396 |
| DS V4 Pro     | 141,565   | 60,005       | 246,685  | 407,202        | 855,457   |
| GPT-5.4       | 143,268   | 66,797       | 233,375  | 388,788        | 832,228   |
| GPT-5.4 Mini  | 141,991   | 66,999       | 217,625  | 372,803        | 799,418   |
| Claude Opus 5 | 322,032   | 153,377      | 667,963  | 996,309        | 2,139,681 |

Worker construction ranges from 1.45 to 4.42 million tokens per model over 100 classes, compared with 0.064–0.149 million for Free Summary construction. This upfront expenditure produces

reusable behavioral records; deterministic prose rendering adds no model call. These totals distinguish the cost of recovering behavior from the cost of generating an executable class. Within each model, the shared workers can be used both with and without a skeleton. The experiment does not compare representations under equal construction budgets.

### F.3 Experiment 2: Construction and Inference

**Table F.16.** CoRe representation construction. The SPL total is shared by Structured SPL and SPL-Prose.

| Construction    | Calls | Input tokens | Output tokens | Total tokens |
|-----------------|-------|--------------|---------------|--------------|
| Free Summary    | 341   | 348,850      | 28,579        | 377,429      |
| Shared SPL      | 2,253 | 4,790,315    | 8,329,821     | 13,120,136   |
| Prose rendering | 0     | 0            | 0             | 0            |

**Table F.17.** CoRe inference usage over 341 tasks. Calls exclude construction.

| Condition      | Calls | Input tokens | Output tokens | Total tokens |
|----------------|-------|--------------|---------------|--------------|
| Raw            | 368   | 1,303,212    | 5,440,689     | 6,743,901    |
| Free Summary   | 370   | 1,353,702    | 5,568,670     | 6,922,372    |
| SPL-Prose      | 341   | 1,485,600    | 4,885,218     | 6,370,818    |
| Structured SPL | 400   | 3,081,693    | 5,330,207     | 8,411,900    |

Structured SPL uses 24.7% more inference tokens than Raw while answering 16 more instances correctly. Total tokens per strict-correct instance are 25.07k for Raw, 25.54k for Free Summary, 23.95k for SPL-Prose, and 29.52k for Structured SPL, where k denotes thousands. Thus the per-correct-instance increase over Raw is 17.7%. These ratios include unsuccessful attempts and exclude construction. Structured SPL uses more input tokens than Raw, while its output usage is slightly lower; the additional input yields a higher inference total. The corresponding mean totals per task are 19,776.84, 20,300.21, 18,682.75, and 24,668.33 tokens in the table’s order. Counting shared worker construction once, construction plus inference is 21,532,036 tokens for the structured workflow considered alone and 19,490,954 for the prose workflow considered alone. Relative to the 6,743,901 inference tokens of Raw, the structured workflow uses about 0.92M additional tokens per additional correct answer. These two standalone totals must not be added to estimate the cost of running both conditions, since that would duplicate preparation.

### F.4 Experiment 3: Inference and Outcome-Normalized Usage

Conditional success among generated patches is 81.4% for Original, 84.3% for Free Summary, 72.7% for Localization, and 75.3% for Repair. These rates concern different submitted-task subsets; all-task resolution remains the primary outcome. The additional patch production includes both newly successful and unsuccessful outcomes, as shown in the main lifecycle figure.

The primary 218-task accounting appears below. The separate 30-task, 80-step accounting is reported once in Table E.13.

*Supplementary outcome-normalized costs.* The main paper reports tokens per resolved task for all six conditions. Mean usage per task for SPL Both and SPL-Prose is 189,503.78 and 606,122.24 tokens, respectively. The four primary means per task are 350,417.17 (Original), 358,248.04 (Free Summary), 196,569.85 (Localization), and 211,246.22 (Repair). All ratios include unsuccessful attempts.

**Table F.18.** Repair inference usage over 218 tasks. The initial representation construction cost is excluded.

| Condition        | Calls | Input tokens | Output tokens | Total tokens |
|------------------|-------|--------------|---------------|--------------|
| Original         | 5,242 | 71,766,439   | 4,624,503     | 76,390,942   |
| Free Summary     | 5,290 | 73,739,087   | 4,358,985     | 78,098,072   |
| SPL Localization | 3,029 | 40,332,826   | 2,519,402     | 42,852,228   |
| SPL Repair       | 3,228 | 43,485,440   | 2,566,235     | 46,051,675   |
| SPL Both         | 3,014 | 38,738,888   | 2,572,937     | 41,311,825   |

SPL Both uses 3,014 model calls, 38,738,888 input tokens, and 2,572,937 output tokens, totaling 41,311,825 inference tokens. SPL-Prose uses 4,138 calls and 132,134,648 tokens. A smaller call count does not imply a smaller token total when each call carries more context. Worker construction for a separate 50-task reference sample used 8,058 calls and 24,581,026 tokens, about 491.6k tokens per task. Construction of the 218-task repair assets was not measured; the 218-task conditions and the 80-step analysis reuse frozen representations. If the 50-task construction sample is representative, the estimated construction cost divided by the per-task inference savings gives token break-even thresholds of approximately 3.2 uses for SPL Localization and 3.5 uses for SPL Repair. Under this estimate, both workflows require at least four uses of the same assets for cumulative token usage to fall below Original. Construction runs on Flash and inference on Pro, so the monetary break-even depends on their price ratio.

For an unchanged representation, reuse can distribute its construction expenditure across subsequent tasks. Updates to the source may require corresponding semantic maintenance.

## F.5 Monetary Accounting

CoRe inference expenditure was approximately \$12.63, \$13.19, and \$13.37 for Raw, Free Summary, and Structured SPL. Repair inference expenditure was \$49.60, \$50.10, \$27.70, \$29.54, and \$26.94 for Original, Free Summary, SPL Localization, SPL Repair, and SPL Both, respectively. These values exclude construction and reflect prices at the time of execution. No new price is inferred for prose controls or gateway model configurations without a verified schedule.

## G Supplementary Representation Controls

The evaluation retains three complementary representation controls. This section records their construction and interpretation in one place.

### G.1 ClassEval SPL-Prose

In RQ1, SPL-Prose renders worker fields as untagged Markdown; in RQ2 and RQ3, it converts them into sentences. For each model arm, the prose representation is generated deterministically from the frozen worker content. It does not invoke a model and does not regenerate program semantics. The tagged and prose conditions therefore share the recovered behavior while presenting it through different representation schemes. On the common 95-task subset, tagged SPL Only is higher in all five model arms; the unadjusted differences are significant for DS Flash, DS Pro, and GPT-5.4 Mini. In Skeleton + SPL, none of the five tagged-versus-prose comparisons is significant. The main table includes full-set PASS and CodeBLEU for SPL-Prose, and Table C.2 gives full-set PASS for Skeleton + SPL-Prose; the method-level table retains partial executable correctness; the paired table above uses the common evaluable subset.

## G.2 CoRe SPL-Prose

The reasoning control uses the exact Free Summary prompt template. A deterministic renderer converts the selected SPL view payload into prose, retaining worker descriptions, input and output contracts, command actions and results, branch conditions, logs, and throws. The content audit verifies retention of 9,005 semantic payload items across 276 parsed views. Prompt leakage checks find zero SPL field markers and zero SPL-protocol phrases in all 341 control prompts.

The control produces 266 correct answers. Its difference from Raw (−3) and Free Summary (−5) is not significant, while its 19-task deficit to structured SPL is significant after the supplementary Holm correction. The 303-task identical-view sensitivity analysis yields 235 prose successes versus 252 structured successes and the same conclusion. These results are used to assess whether richer semantic content, when supplied as an ordinary summary, reproduces the structured workflow.

## G.3 SWE-bench SPL-Prose

For each repair task, every function entry from the frozen SPL index is rewritten into ordinary sentences and appended once to the Original agent template. The conversion retains file and function names as ordinary text but does not expose worker blocks or task-driven worker access. No model is used to regenerate the semantic material. The appended context has a median of 10,290 tokens, mean 17,576, and maximum 104,889.

**Table G.19.** Semantic inputs, consumption formats, and outcomes of the three supplementary representation controls. All conversions use frozen semantic assets and require no additional model calls.

| Control             | Preserved semantic source       | Ordinary-language use                             | Observed result                                                                      |
|---------------------|---------------------------------|---------------------------------------------------|--------------------------------------------------------------------------------------|
| ClassEval SPL-Prose | Frozen SPL worker content       | Untagged Markdown rendering, no semantic rebuild  | Tagged SPL Only higher in 5/5 model arms; Skeleton + SPL differences not significant |
| CoRe SPL-Prose      | Selected SPL view payload       | Exact Free Summary prompt framework               | 266/341 versus 285/341 structured SPL                                                |
| SWE-bench SPL-Prose | All entries in frozen SPL index | One prose block appended to Original agent prompt | 100/218 resolved, 120 patches, 132.13M inference tokens                              |

## H Additional Case Evidence

The cases below are selected illustrations from the repair trajectories. Outcome claims refer to evaluated patches; explanatory prose summarizes the relation between the source, semantic evidence, and patch.

*Output roles in Q.deconstruct.* The motivation case, `django__django-14140`, contrasts a defensive guard with removal of the single child branch. Original remains unresolved and the three SPL variants resolve the task. The figure uses Original and SPL Repair to avoid combining insertion strategies. Its worker is an abbreviated description of the implementation before repair. Its MAIN\_FLOW describes the ordinary path, and its ALTERNATIVE\_FLOW retains the single non-Q child branch. That branch still assigns the child to a different output role. The figure’s Code Only and Code + SPL labels correspond to Original and SPL Repair, respectively. The desired uniform role is obtained by relating this observation to the issue, not by silently correcting the description during construction.

**Table H.20.** Representative RQ3 repair cases.

| Instance             | Observed Outcomes                                           | What the Case Shows                                                                                                                         |
|----------------------|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| django__django-12858 | Original / Summary: empty patch; all SPL variants: resolved | The SPL trajectories reach an evaluated edit within the interaction limit; the baseline leaves no patch.                                    |
| django__django-15037 | Original: empty patch; Free Summary: resolved               | The summary condition resolves an instance left without a patch by Original, illustrating that free prose can also provide useful guidance. |

*Creation effects and return values in formsets.* For django\_\_django-14725, the requested edit only mode must prevent new database objects while allowing updates to existing objects. The worker for save\_new\_objects identifies the initialization of self.new\_objects, iteration over extra forms, object creation, and return of the resulting list. These facts connect the requested restriction to an early return after list initialization and before creation. The patch also propagates the option through modelformset\_factory and inlineformset\_factory. Original leaves no patch, whereas Localization and Repair resolve the task. SPL Both also resolves the task.

*A framework method in SymPy.* The SymPy case, sympy\_\_sympy-23950, concerns the method Contains.as\_set. The source-only patch returns the set operand but renames the method to \_eval\_as\_set, which the relevant call path does not use. SPL Repair retains as\_set and returns self.args[1], resolving the task. This case illustrates the importance of associating a semantic responsibility with the concrete function through which callers reach it. A plausible behavior in the wrong extension point does not satisfy the required call behavior.

*Preserving decorator metadata.* For django\_\_django-14787, both Original and SPL Repair resolve the task. The issue concerns a decorator receiving a bound callable without the expected metadata. The generated patch copies metadata onto that callable before applying the decorator; copying metadata only onto the outer wrapper occurs too late for this use. Repair uses 88,931 inference tokens versus 131,190 for Original, a 32.2% reduction. This illustrates an instance level saving, not a universal reduction for successful repairs.

## I Representative Prompts

This section presents a selected subset of the prompts used in Code2SPL, covering two construction components and the three evaluated downstream tasks. The construction examples comprise function-call analysis and schema-constrained worker generation: the former shows how callee references are introduced during semantic analysis, while the latter shows how the recovered information is organized into a structured representation. The usage examples comprise Skeleton + SPL class reconstruction, Structured SPL static reasoning, and SPL Repair, illustrating how workers guide implementation recovery, source-based dependence analysis, and issue-driven modification, respectively. These five examples are selected to trace the connection from semantic construction to task-specific use and expose the instructions governing reference preservation, interface compatibility, source verification, and repair. The generation prompt is excerpted; complete templates and runtime prompt-assembly code are available in the replication package.

The retained prompt text follows the repository templates. The generation prompt is excerpted to omit the detailed JSON schema; the marker [...] denotes an editorial omission and is not part of the executed prompt. Placeholders in braces are populated at runtime. The prompts use

the implementation’s term “card” for a view. Complete templates, schemas, and prompt-assembly code are provided in the replication package.

### I.1 Function-Call Semantic Analysis

The call analyzer examines a source fragment containing a function call. In addition to describing argument propagation and the call’s role, it preserves an explicit reference when the callee has an SPL entry. This reference can subsequently be retained in the generated caller worker. The example comes from `function_call_analyzer.txt` and illustrates the call-analysis component of semantic construction.

#### Prompt I.1: Function-Call Semantic Analysis

```
[DEFINE_AGENT: FunctionCallAnalyzer]
[DEFINE_PERSONA:]
Role: Function-call analysis expert specializing in concise analysis of function calls and
argument propagation.
Style: concise and direct.
[END_PERSONA]

[DEFINE_VARIABLES:]
_CALL_CODE: text "Function-call code to analyze"
_CALLED_FUNCTION_NAME_IF_SPL: text "Called function name if it belongs to the analyzed input
set and has an SPL entry; otherwise empty"
_CALL_ANALYSIS: text "Concise function-call analysis"
[END_VARIABLES]

[DEFINE_WORKER: AnalyzeFunctionCall]
[INPUTS]
<REF> _CALL_CODE </REF>: text
<REF> _CALLED_FUNCTION_NAME_IF_SPL </REF>: text
[END_INPUTS]

[OUTPUTS]
<REF> _CALL_ANALYSIS </REF>: text
[END_OUTPUTS]

[MAIN_FLOW]
[SEQUENTIAL_BLOCK]
[COMMAND "Identify the called function and module. If _CALLED_FUNCTION_NAME_IF_SPL is non-
empty, write that function name as <SPL>_CALLED_FUNCTION_NAME_IF_SPL</SPL> in the output"
RESULT _function_identification]
[COMMAND "Analyze how arguments are passed and what values or objects flow into the call"
RESULT _argument_analysis]
[COMMAND "Understand the purpose of the call in the current code fragment" RESULT
_purpose_analysis]
[COMMAND "Generate a concise call analysis. If you mention the called function name and
_CALLLED_FUNCTION_NAME_IF_SPL is non-empty, preserve the exact <SPL>...</SPL> wrapper. Do
not output irrelevant content" RESULT <REF> _CALL_ANALYSIS </REF>]
[END_SEQUENTIAL_BLOCK]
[END_MAIN_FLOW]
```

```
[DEFINE_CONSTRAINTS]
- When _CALLED_FUNCTION_NAME_IF_SPL is non-empty, every mention of that called function in the
  analysis output must use the format <SPL>function_name</SPL>.
- Only omit the <SPL> tag when _CALLED_FUNCTION_NAME_IF_SPL is empty.
[END_CONSTRAINTS]
[END_WORKER]
[END_AGENT]
```

## 1.2 Schema-Constrained Worker Generation

The generator receives the method source, the target template, and analyses of its constituent fragments. It produces a JSON representation that is subsequently rendered into an SPL worker. The required fields include the worker name, a brief description, and the main flow; optional fields describe inputs, outputs, alternative flows, and exceptional flows.

The following excerpt from `method_spl_agent.txt` retains the generation instructions and semantic-preservation rules. These rules address constants, condition boundaries, nested control structures, early returns, short-circuit evaluation, and exceptional paths. They also require callee-reference markers to survive generation. The declarations of the JSON schema are omitted from the box.

### Prompt 1.2: Schema-Constrained Worker Generation (Excerpt)

```
[DEFINE_AGENT: MethodToTemplateConverter]
[DEFINE_PERSONA:]
Role: Code conversion expert. Convert a method into a strict JSON representation that can be
deterministically rendered into SPL, preserving full semantics and important boundary
details.
[END_PERSONA]
<!-- Focus on semantic completeness and stable machine-parseable output -->
[DEFINE_VARIABLES:]
_INPUT_TEMPLATE: text "Target template structure"
_INPUT_CODE: text "Method code to convert"
_SUB_ANALYSIS_RESULTS: list "Sub-code analysis results"
_OUTPUT: text "Final JSON string"
[END_VARIABLES]

[DEFINE_WORKER: ConvertMethodToTemplate]
[INPUTS]
<REF> _INPUT_TEMPLATE </REF>: text
<REF> _INPUT_CODE </REF>: text
<REF> _SUB_ANALYSIS_RESULTS </REF>: list "List of sub-code analyses. Example format: [{ 'type':
  'For', 'code_snippet': 'for i in range(10):', 'analysis_result': '...' }]"
[END_INPUTS]

[OUTPUTS]
<REF> _OUTPUT </REF>: text
[END_OUTPUTS]

[MAIN_FLOW]
[SEQUENTIAL_BLOCK]
```

```

[COMMAND "Understand the target SPL template structure but do not output any SPL text" RESULT
_template_understanding]
[COMMAND "Understand the full method semantics from the input code and all sub-analyses; do
not lose meaningful behavior. IMPORTANT: If any sub-analysis text contains <SPL>...</SPL>,
treat it as a required reference marker and preserve it verbatim in any final command,
condition, or log text that mentions that function name" RESULT _method_understanding]
[COMMAND "Derive a granular step plan where one step equals one coherent process; preserve
numeric constants, condition boundaries, early returns, short-circuits, and exception
paths. IMPORTANT: When describing a step that calls a referenced internal function, keep
the function name wrapped exactly as <SPL>...</SPL> if it appears that way in sub-
analyses" RESULT _step_plan]
[COMMAND "Generate brief_description as one single English sentence summarizing the method's
primary purpose and key transformation; it must be a real summary of this code, not
placeholder text" RESULT _brief_summary]
[COMMAND "Construct a strict JSON object following the schema rules; omit optional sections if
they do not exist. IMPORTANT: In JSON fields main_flow[].command, alternative_flows[.
steps[].command, exception_flows[.condition, and log, preserve any <SPL>...</SPL>
wrapper exactly when mentioning that function" RESULT _json_draft]
[COMMAND "Output only the JSON object as the final answer; no markdown, no comments, no
surrounding text" RESULT <REF> _OUTPUT </REF>]
[END_SEQUENTIAL_BLOCK]
[END_MAIN_FLOW]

[...]

Rules:
1) Output must be valid JSON, no trailing commas, no extra keys.
2) English only for all descriptions and command texts.
3) Do not output any SPL tokens such as [DEFINE_WORKER] inside JSON values.
4) Each main_flow item must be one step per line level; do not embed lists inside a command
string.
5) Preserve semantics that change behavior: numeric constants, inclusive/exclusive boundaries,
early returns, short-circuit logic, and exception paths.
6) When a step belongs to a nested control structure, include the full nesting context so the
hierarchy is recoverable. Example: "Within the outer for loop over i, for each j from 1
to t, compute dp[i][j]."
7) Only use exception_flows when the code explicitly throws or raises an exception. If the
code catches an exception and returns a default value or falls back to alternative logic,
describe that path in main_flow or alternative_flows instead.
8) brief_description must be a real summary of the specific method, not fixed text from the
template.

[...]

[END_WORKER]
[END_AGENT]

```

### I.3 Class Reconstruction with Skeleton and SPL

The Skeleton + SPL condition assigns complementary roles to its two inputs. The ClassEval skeleton supplies the concrete interface, while workers describe the implementation behavior. The

prompt instructs the model to associate workers with methods, trace value and callee references, and reconstruct method bodies from commands and execution paths. If the inputs disagree about the interface, the skeleton takes precedence.

The following template is used by the `skeleton_spl` condition. The placeholders `{skeleton}` and `{spl_inline}` are replaced with the class skeleton and its per-function views.

**Prompt I.3: Class Reconstruction with Skeleton + SPL**

Task:

Complete the Python class from the `ClassEval` skeleton, using SPL as a structured implementation contract.

Skeleton Role:

- The skeleton controls imports, class name, constructor shape, field names, method names, and method signatures.
- Preserve the skeleton API if skeleton and SPL disagree.

SPL Concept:

- SPL is a code-derived structured natural-language view of the same class.
- Each inline card pairs a skeleton method with its SPL ``[DEFINE_WORKER]`` block, so API shape and implementation semantics are shown together.
- ``[INPUTS]`` and ``[OUTPUTS]`` describe method contracts.
- ``[COMMAND]``, ``[CONDITION]``, ``[RETURN]``, ``[ALTERNATIVE_FLOW]``, and ``[EXCEPTION_FLOW]`` describe implementation checkpoints.
- ``<SPL>name</SPL>`` references another method with its own worker block.
- ``<REF>name</REF>`` marks a variable, parameter, field, or intermediate value reference.

SPL Reconstruction Workflow:

Step 1: Match each SPL card to its skeleton method by method name.

Step 2: Treat the skeleton as the public API contract and SPL as the implementation contract for method bodies.

Step 3: Build a class-level behavior map:

- constructor fields and shared state;
- method inputs, outputs, and side effects;
- cross-method calls from ``<SPL>`` references;
- data flow from ``<REF>`` references and `COMMAND RESULT` values;
- branch and exception behavior from alternative and exception flows.

Step 4: Implement each method from the mapped checkpoints:

- translate ``[MAIN_FLOW]`` `COMMAND` steps into Python statements in order;
- convert ``[ALTERNATIVE_FLOW]`` branches into `if/elif/else` blocks;
- convert ``[EXCEPTION_FLOW]`` into `try/except` or `explicit raise` only when entailed;
- use ``<REF>`` chains to choose local variables and returned values;
- use ``<SPL>`` call references to preserve method dependencies.

Step 5: Keep the implementation minimal and executable. Do not add unrelated APIs or behavior outside the skeleton plus SPL contract.

Reconstruction Rules:

- Fill in method bodies and shared-state behavior using the skeleton plus SPL.
- Preserve cross-method dependencies, field usage, return values, and error behavior described by SPL.
- Output only the complete Python class implementation.
- Do not use markdown fences and do not explain.

ClassEval skeleton:  
{skeleton}

Per-Function SPL Cards:  
{spl\_inline}

#### 1.4 Static Reasoning with Structured SPL

The static-reasoning template retains the benchmark’s official question and output requirements. It adds a task-specific analysis control and function views containing source and SPL. The control is assembled at runtime from the question, selected functions, and source anchors; it is represented below by {spl\_control}.

The template directs the consumer to verify semantic descriptions against numbered source code and apply the benchmark’s dependence definitions. Its information-flow instructions require closure in both directions around the queried variable instance. This example is the static template of the raw\_spl\_atomic\_strict condition; dynamically generated controls and any retry prompts are supplied by the accompanying implementation.

##### Prompt I.4: Static Reasoning with Structured SPL

{official\_prompt}

## SPL source-bound analysis control

{spl\_control}

## Atomic function source + SPL cards

{spl\_inline}

Follow the official dependence definition exactly. The SPL cards are an auto-generated semantic description of the code and are evidence for its intended behavior, but they are not guaranteed correct; when an SPL description and the numbered source disagree, the source is the ground truth. Derive every reported variable instance and line number from the numbered source code.

For information flow, compute the complete bidirectional dependence component around the queried instance. Do not report a query instance unless that numbered line defines or updates it, and do not report an object used only as a method-call receiver unless the object’s value itself participates in the computed flow. A receiver initialized from another object’s data-returning call carries that returned value and must remain in the flow; a root I/O handle initialized only from an external stream does not.

Return only the exact JSON object required by the official prompt, with no prose or markdown.

1.5 Repository-Level Repair with SPL Workers

The repair example is the condition-specific addendum for SPL Repair, checked against its frozen copy. It is appended to the mini-SWE-agent instance template. The agent also receives the issue and an embedded patch-production plan, which are not reproduced here.

The addendum connects semantic guidance to source inspection and editing. It directs the agent to compare the requested behavior with the current implementation, identify the source construct responsible for the discrepancy, and preserve adjacent behavior. Workers describe the existing implementation; the issue supplies the desired change. The final step records the condition’s validation and submission procedure.

Prompt I.5: SPL Repair Workflow

## SPL Repair Workflow

SPL is a source-derived structured behavior map. In this variant it should make repair faster by pointing to the likely current-behavior owner and preservation checks. The editable source code is the only patch target.

Step 1: Follow the embedded SPL Patch Production Plan first. Run its first source command and inspect the real source owner before broad search. Only open ``tmp/spl_tools/source_bound_cards.md`` if the embedded plan is insufficient.

Step 2: Compare the issue requirement with the visible source owner. Use the SPL checkpoint to focus on the concrete source construct that may need changing: condition, return, assignment, call, parser/formatter rule, exception path, or data-flow edge.

Step 3: Patch the smallest source construct that owns the conflicting behavior. Prefer a narrow named function, expression, branch, return, assignment, or formatter/printer handler. Do not move to a helper, caller, constructor, wrapper, broad fallback handler, dispatcher, or semantic neighbor unless the visible source proves it directly controls the failing behavior.

Step 4: Use SPL only to preserve adjacent behavior after choosing the source edit. Never copy SPL text, tags, summaries, or natural-language steps into the patch.

Step 5: After editing, run at most one issue-specific sanity check. If it passes, stop searching immediately, create and inspect ``patch.txt``, then submit in the next action. Do not run edge-case sweeps, broad test suites, or extra dispatcher investigations after a passing check. Every response must contain exactly one ``mswea_bash_command`` block; never answer with analysis text only.